

Software Requirements Specification (SRS)

AI Chatbot System

1. Project Overview

1.1 Project Name

AI Chatbot Assistant

1.2 Purpose

The purpose of this project is to develop an AI-powered chatbot that provides intelligent, context-aware, and human-like responses to user queries. The chatbot will assist users in obtaining information, answering questions, providing recommendations, and automating support interactions.

1.3 Scope

The chatbot will:

- Accept user queries through text input.
- Generate intelligent responses using Large Language Models (LLMs).
- Maintain conversational context within a session.
- Support document-based question answering.
- Store chat history.
- Provide fast and accurate responses.
- Integrate with external APIs and databases when required.

2. Stakeholders

Role	Responsibility
Client	Defines business requirements
Project Manager	Project planning and monitoring
UI/UX Team	User interface design
Frontend Team	User-facing application
Backend Team	APIs and business logic
AI/ML Team	LLM integration and optimization

Role	Responsibility
QA Team	Testing and validation
End Users	Use chatbot services

3. Functional Requirements

FR-01 User Authentication (Optional)

Description

Users may log in to access personalized chat history.

Acceptance Criteria

- User can register.
 - User can log in.
 - User can reset password.
-

FR-02 Chat Interface

Description

Users should interact with the chatbot through a simple messaging interface.

Features

- Message input box.
- Send button.
- Chat history display.
- Typing indicator.
- Response streaming.

Acceptance Criteria

- User can send text messages.
 - Chatbot displays responses instantly.
 - Conversation appears in chronological order.
-

FR-03 AI Response Generation

Description

Generate responses using an LLM.

Inputs

- User message.
- Conversation history.

Outputs

- Context-aware response.

Acceptance Criteria

- Response generated within 5 seconds.
 - Response relevance above 90%.
 - Handles follow-up questions.
-

FR-04 Conversation Context Management

Description

Maintain context during active sessions.

Acceptance Criteria

- Bot remembers previous messages.
 - Context retained throughout session.
 - Handles multi-turn conversations.
-

FR-05 Chat History Management

Description

Store and retrieve previous conversations.

Acceptance Criteria

- User can view past chats.
- Chats stored securely.
- User can delete conversations.

FR-06 Document-Based Question Answering

Description

Users can upload documents and ask questions related to them.

Supported Formats

- PDF
- DOCX
- TXT

Acceptance Criteria

- Documents processed successfully.
 - Answers generated from uploaded content.
 - Multiple documents supported.
-

FR-07 Feedback System

Description

Users can rate chatbot responses.

Acceptance Criteria

- Like/Dislike button available.
 - Feedback stored in database.
 - Analytics available to admin.
-

FR-08 API Integration

Description

Chatbot can fetch information from external services.

Examples

- Weather APIs
- Knowledge APIs
- Company databases

Acceptance Criteria

- APIs respond successfully.
 - Error handling implemented.
 - Timeout management supported.
-

FR-09 Admin Dashboard

Description

Administrative panel for monitoring chatbot performance.

Features

- Total users.
- Total conversations.
- Feedback analytics.
- Usage reports.
- Error monitoring.

Acceptance Criteria

- Dashboard updates in real-time.
 - Admin-only access.
-

4. Non-Functional Requirements

Performance

- Response time < 5 seconds.
- Support 1,000+ concurrent users.
- 99.5% uptime.

Security

- HTTPS communication.
- JWT Authentication.
- Encrypted user data.
- Secure API keys.

Scalability

- Horizontal scaling support.

- Cloud deployment readiness.

Reliability

- Automatic recovery on failure.
- Error logging and monitoring.

Usability

- Mobile responsive UI.
 - Simple and intuitive interface.
-

5. System Architecture

Frontend

- React.js / Next.js

Backend

- Node.js + Express OR
- FastAPI

Database

- MongoDB / PostgreSQL

AI Layer

- OpenAI GPT
- Llama
- Gemini

Document Processing

- LangChain
- Vector Database (ChromaDB/Pinecone)

Deployment

- Docker
 - AWS / Azure / GCP
-

6. User Flow

1. User opens chatbot.
 2. User enters query.
 3. Request sent to backend.
 4. Backend sends prompt to AI model.
 5. AI generates response.
 6. Response displayed to user.
 7. Conversation stored in database.
 8. User provides feedback (optional).
-

7. Database Requirements

Users Collection

Fields:

- userId
- name
- email
- password
- createdAt

Conversations Collection

Fields:

- conversationId
- userId
- messages
- createdAt
- updatedAt

Feedback Collection

Fields:

- feedbackId
 - userId
 - messageId
 - rating
 - comments
-

8. Testing Requirements

Functional Testing

- Message sending.
- Response generation.
- Context retention.
- Authentication.

Performance Testing

- Concurrent users.
- Response latency.

Security Testing

- Authentication.
- Authorization.
- Data protection.

Integration Testing

- AI model integration.
 - Database integration.
 - External API integration.
-

9. Deliverables

1. Frontend Application
 2. Backend APIs
 3. AI Integration Module
 4. Database Setup
 5. Admin Dashboard
 6. Deployment Configuration
 7. Test Cases
 8. Technical Documentation
 9. User Manual
-

10. Success Criteria

- Response accuracy $\geq 90\%$.

- User satisfaction $\geq 4/5$.
- Response time < 5 seconds.
- Successful handling of 1,000+ concurrent users.
- Secure and scalable production deployment.